Evaluating and Refining Arch-Stage

Supporting Architectural Hypotheses in Software Engineering Teams

José Gonçalves Lima Neto

Thesis presented to the
Institute of Mathematics,
Statistics, and Computer Science
of the University of São Paulo
in partial fulfillment
of the requirements
for the degree of
Master of Science

Program: Computer Science
Advisor: Prof. Dr. Paulo Roberto Miranda Meirelles

São Paulo
December, 2026

Evaluating and Refining Arch-Stage

Supporting Architectural Hypotheses in Software Engineering Teams

José Gonçalves Lima Neto

This is the original version of the thesis prepared by candidate José Gonçalves Lima Neto, as submitted to the Examining Committee.

The content of this work is published under the CC BY 4.0 license
(Creative Commons Attribution 4.0 International License)

Acknowledgments

Abstract

José Gonçalves Lima Neto. **Evaluating and Refining Arch-Stage: *Supporting Architectural Hypotheses in Software Engineering Teams***. Thesis (Master's). Institute of Mathematics, Statistics, and Computer Science, University of São Paulo, São Paulo, 2026.

Software engineering teams frequently face uncertainties related to their systems' architecture yet lack adequate methods and tools to manage them explicitly and collaboratively. The ArchHypo technique proposes using hypotheses engineering to make such uncertainties visible and manageable; however, its manual application imposes a significant learning curve. This work empirically evaluates and refines Arch-Stage, a support tool for the ArchHypo technique, by applying it with multiple software engineering teams in a real-world organization with a scaled software product. The research adopts a mixed-methods design combining analysis of artifacts produced in the tool, questionnaires, and interviews with team members. Preliminary results indicate how the tool supports the management of architectural uncertainties and inform its ongoing improvement.

Keywords: Software Architecture. Architectural Uncertainty. Hypotheses Engineering. ArchHypo. Arch-Stage. Empirical Evaluation.

Resumo

José Gonçalves Lima Neto. **Avaliação e Refinamento do Arch-Stage:** ***Apoio a Hipóteses Arquiteturais em Equipes de Engenharia de Software***. Dissertação (Mestrado). Instituto de Matemática, Estatística e Ciência da Computação, Universidade de São Paulo, São Paulo, 2026.

Equipes de engenharia de software frequentemente enfrentam incertezas relacionadas à arquitetura de seus sistemas, mas carecem de métodos e ferramentas adequadas para gerenciá-las de forma explícita e colaborativa. A técnica ArchHypo propõe o uso de engenharia de hipóteses para tornar essas incertezas visíveis e gerenciáveis, porém sua aplicação manual impõe uma curva de aprendizado significativa. Este trabalho avalia e refina empiricamente o Arch-Stage, uma ferramenta de apoio à técnica ArchHypo, por meio de sua aplicação com múltiplas equipes de engenharia de software em uma organização real com um produto em escala. A pesquisa adota um design de métodos mistos, combinando análise de artefatos produzidos na ferramenta, questionários e entrevistas com membros das equipes. Os resultados preliminares indicam como a ferramenta apoia o gerenciamento de incertezas arquiteturais e informam seu aprimoramento contínuo.

Palavras-chave: Arquitetura de Software. Incerteza Arquitetural. Engenharia de Hipóteses. ArchHypo. Arch-Stage. Avaliação Empírica.

List of Abbreviations

SA	Software Architecture
ArchHypo	Architecture Hypothesis (technique)
IDP	Internal Developer Portal
NFR	Non-Functional Requirements
QA	Quality Attribute
DSR	Design Science Research
HE	Hypotheses Engineering
IME	Institute of Mathematics and Statistics (Universidade de São Paulo)
USP	University of São Paulo
ADR	Architecture Decision Record

List of Figures

List of Tables

List of Programs

Contents

1 Introduction **1**

1.1 Architectural Uncertainty and Hypotheses Engineering 2
1.2 From Requirements to Architecture and the Role of Tool Support 3
1.3 Problem Statement 4
1.4 Objectives 4
1.5 Research Questions 5
1.6 Research Contributions 5
1.7 Research Structure 6

2 Background **7**

2.1 Software Architecture and Architectural Uncertainty 7
2.2 Hypotheses Engineering and ArchHypo 8
2.2.1 Core Concepts 8
2.2.2 Assessment 9
2.2.3 Technical Plans 9
2.2.4 Lifecycle 10
2.2.5 Empirical Evaluations 10
2.3 Arch-Stage: The Initial Tool 11
2.4 Deriving Architecture from Requirements: The Tooling Gap 12

3 Related Work **15**

3.1 Centralization and Decentralization of Architectural Decisions 15
3.2 Tool Support for Architectural Decision-Making 16
3.3 Empirical Studies of Architectural Practices in Industry 17

4 Research Design **19**

4.1 Empirical Context 19
4.2 Research Approach 19
4.3 Data Sources 20

4.4 Analysis Plan 21
4.5 Ethical Considerations 21
4.6 Alignment with Qualification Feedback 21

Appendixes

A Research Instruments **23**
A.1 Questionnaire 23
A.2 Interview Guide 23
A.3 Informed Consent Form 23

References **25**

Chapter 1

Introduction

Software architecture (SA) plays a central role in the development and evolution of software-intensive systems. As firstly conceived by Perry and Wolf, 1992, the SA concept structures the system into elements and relations that form the basis of the system's organization as it must convey cohenrence, scalability, and maintainability for the relaying software over time. Despite the ever-changing practices in the modern era of software development, SA still provides the basis for analyzing the multiple quality attributes of nowadays software systems as it guides technical decisions that must be made throughout the development process over time via a diverse set of archictetual methodologies as the ones systematically mapped by Souza *et al.*, 2019. In organizations that rely on software as a core part of their business, architecture is not just a simple one-time design artifact, but a continuous decision-making process that must accommodate changing requirements, evolving technologies, and critical organizational constraints K. Silva, J. Melegati, *et al.*, 2025.

In recent years, agile and continuous development practices have intensified this fast-paced pressure. Teams are expected to deliver more features in shorter cycles while preserving scalability, performance, and other quality attributes of their systems that continue to grow in size and complexity K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024. In such environments, architecture decisions are frequently made under uncertainty: solutions or requirements may be incomplete or unstable, the impact of technology choices may not be fully understood, and the behavior of the system under realistic workloads may only become visible after deployment K. Silva, J. Melegati, *et al.*, 2025. Despite this, architectural uncertainties often remain implicit and unmanaged, and decisions tend to rely on the tacit knowledge and intuition of a small group of experienced architects as oberseved by Souza *et al.*, 2019.

At the same time, many organizations seek to organize themselves around autonomous, cross-functional teams responsible for end-to-end delivery of features. Recent industry reports, such as the 2024 InfoQ Software Architecture and Design Trends Report Betts *et al.*, 2024, highlight socio-technical architecture as an emerging trend that emphasizes the co-design of technical and organizational structures, and the importance of considering how team boundaries and communication patterns shape system architectures. In this view,

architecture work is distributed across teams rather than centralized in a separate function or role. Moreover, in these scenarios, the Conway's Law is treated as a design constraint to be intentionally leveraged rather than an accident to be ignored Betts *et al.*, 2024. However, when architecture decisions depend on implicit assumptions held by a few experts in the organization, this distribution becomes difficult in practice. Less experienced team members may struggle to participate meaningfully in architectural discussions, and teams may find it hard to reason about the risks and trade-offs of alternative solutions even when they are applying anyone of thewell-known and validated architectural methodologies mapped by Souza *et al.*, 2019. This challenge reveals the case of the architectural uncertainty management, which still is a critical aspect of the architecture work, but is often neglected or poorly supported in practice by the available tools and methodologies.

1.1 Architectural Uncertainty and Hypotheses Engineering

Uncertainty in architecture decisions stems from limited knowledge about requirements, future usage scenarios, and the properties of candidate solutions as reviewed in K. Silva, J. Melegati, *et al.*, 2025. Traditional risk management techniques assume that relevant events and their probabilities are known or can be estimated, which is not always the case when dealing with evolving systems and emerging technologies K. Silva, J. Melegati, *et al.*, 2025. A complementary perspective is to acknowledge uncertainty explicitly and as an inevitable aspect of the architecture work, and therefore, to manage it as a first-class concern during the architecture work as proposed in K. Silva, J. Melegati, *et al.*, 2025.

In this regard, Hypotheses Engineering has emerged as an approach to manage uncertainty in software development by treating assumptions as explicit, falsifiable hypotheses that can be prioritized and tested through experiments K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024. In this context, ArchHypo is the technique that adapts hypotheses engineering to software architecture. It proposes that teams identify *architectural hypotheses* — statements capturing uncertainties that may affect architectural decisions — assess each hypothesis by its uncertainty level and impact, and define a technical plan for handling it K. Silva, J. Melegati, *et al.*, 2025.

The technical plan may include actions to reduce uncertainty, such as experiments, architectural spikes, or tracer bullets, and actions to reduce the impact of potential changes, such as modularization or flexible deployment strategies K. Silva, J. Melegati, *et al.*, 2025. By iteratively updating hypotheses and technical plans over time, ArchHypo aims to support a more deliberate and distributed approach to architectural decision-making, allowing teams to postpone irreversible decisions when justified and to focus their efforts on the most critical uncertainties K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024.

Empirical studies have started to explore how ArchHypo works in practice. An experience report in a software start-up describes how the technique helped the founder identify and prioritize architectural uncertainties related to availability, usability, and scalability over a ten-month period, guiding the evolution of the product and avoiding premature investments in infrastructure K. Silva, J. A. Melegati, *et al.*, 2024. A design science

study in a mission-critical public-sector system shows that integrating ArchHypo into the development process enabled the team to distribute architectural work across iterations and to make better-informed decisions in the face of changing requirements K. Silva, J. Melegati, *et al.*, 2025. At the same time, both studies report that the technique introduces a non-trivial learning curve and requires adjustments to existing processes, which might pose a challenge to its adoption by the team as reported in K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024.

1.2 From Requirements to Architecture and the Role of Tool Support

The challenges of managing architectural uncertainty are closely related to the long-standing problem of deriving architectural models from requirements specifications. A systematic mapping study by Souza *et al.*, 2019 reviewed 39 methods for this derivation and concluded that existing approaches still rely heavily on the tacit knowledge of architects, provide limited support for less experienced practitioners, and often lack explicit evaluation mechanisms. Many methods describe high-level processes or frameworks, but offer little guidance on how to operationalize them in everyday practice as reported in Souza *et al.*, 2019.

The same study highlighted gaps in tool support. A significant portion of the methods did not include any supporting tool, and when tools were present, they tended to address only parts of the process as also reported in Souza *et al.*, 2019. The authors called for more automation, better mechanisms to capture and reuse architectural knowledge, and empirical validation of tools in realistic settings. These findings resonate with the observations from ArchHypo studies: while the technique provides a structured way to make architectural uncertainties explicit and to plan actions, applying it in practice is highly demanding without adequate supportting tool.

Grounded in these studies' insights, Corrêa, 2025 developed the Arch-Stage, a support tool designed to help teams apply the ArchHypo technique in practice. Up to a specific point in its history, the work focused on designing and implementing the core functionality for enabling ArchHypo capabilities, such as representing architectural hypotheses, recording their uncertainty and impact assessments, and managing associated technical plans.[1] The work also discussed initial design decisions and limitations, including usability aspects and open questions about how the tool would fit into different team workflows. A more detailed description of Arch-Stage and the contributions by Corrêa, 2025 are presented in Chapter 2.

[1] The first version of Arch-Stage and the corresponding source code history up to commit `ceee509776508081ebdd473c2c4f710b8ef55947` document the initial design and implementation of Arch-Stage proposed by Corrêa, 2025.

1.3 Problem Statement

When reviewed in intersection, these studies outline a clear problem. On the one hand, modern software organizations need to manage architectural uncertainty in a way that supports continuous evolution and involves the broader development team K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024; Betts *et al.*, 2024. On the other hand, empirical evidence and systematic mapping studies indicate that:

- Methods for deriving and evolving architectures from requirements still depend strongly on architects' tacit knowledge and lack comprehensive tool support as reported in Souza *et al.*, 2019.
- ArchHypo offers a promising structure for handling architectural uncertainty, but teams perceive the technique as hard to learn and requiring process changes, especially without specialized support as reported in K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024.
- Arch-Stage provides an initial implementation of tool support for ArchHypo, but its impact on real software engineering teams and the refinements needed for effective use in industry remain unknown as reported in Corrêa, 2025.

In this context, there is a need to understand how software engineering teams, with diversed roles, tenures and responsibilities, interact with a tool like Arch-Stage when working on their products and services. By forging this path, we seek to gain an understanding of what sorts of architectural uncertainties they manage through, and which aspects of the Arch-Stage tool and its integration into the development process still require refinement. Addressing these concerns can contribute both to the practical adoption of ArchHypo via Arch-Stage tool and to the broader discussion on how to support teams in sharing architectural responsibilities for accelerating adoption of the socio-technical architecture paradigm mapped in Betts *et al.*, 2024.

1.4 Objectives

The overall objective of this present work is to empirically evaluate and refine the Arch-Stage tool, a support tool for the ArchHypo technique from Corrêa, 2025, by applying it with diverse software engineering teams in a real-world organization that maintains a scaled software product in Latin America. More specifically, this work aims to:

- Investigate how software engineering teams appropriate and use Arch-Stage and ArchHypo in their day-to-day activities when working on an existing, scaled product.
- Derive and implement refinements to Arch-Stage and its usage guidelines based on empirical findings, with the goal of improving its suitability for supporting architectural work in teams.
- Characterize the architectural uncertainties that teams make explicit and manage through Arch-Stage, and how these uncertainties evolve over time.
- Identify benefits and challenges perceived by team members regarding the use of

Arch-Stage, including its learning curve, usability, and fit with existing practices.

These objectives deliberately build on, rather than replicate, the previous studies' contributions in the ArchHypo research line. While the Corrêa, 2025 focused on designing and implementing Arch-Stage, this work treats the tool as an existing artifact and concentrates on its evaluation and evolution in an industrial context.

1.5 Research Questions

To achieve the above objectives, this work is guided by the following research questions. At this stage, the exact emphasis between tool interaction and uncertainty patterns is still under refinement; therefore, the questions are formulated to accommodate both perspectives, and later chapters will make their final scope explicit.

RQ1: How do software engineering teams use Arch-Stage and the ArchHypo technique when working on a scaled software product in a real-world organization?

RQ2: Which refinements to Arch-Stage and its usage guidelines emerge from the empirical study, and how do they address the identified challenges?

RQ3: What types of architectural uncertainties do these teams identify, record, and manage using Arch-Stage over the course of their work?

RQ4: What benefits and challenges do team members perceive in using Arch-Stage to support architectural decision-making and uncertainty management?

As the empirical design is detailed in Chapter 4, these questions may be operationalized with different emphases — for example, with RQ1 and RQ3 focusing more strongly on interaction and process, and RQ2 providing a descriptive view of uncertainty patterns — to accommodate the different research questions' emphases.

1.6 Research Contributions

By answering its set of research questions, this work attempts to make the following contributions:

- An empirical account of how multiple software engineering teams in a real organization use a tool-supported implementation of ArchHypo (via Arch-Stage tool) while working on a scaled software product, including observed practices, adaptations, and obstacles as reported in Corrêa, 2025.
- A set of empirically grounded refinements to the Arch-Stage tool and its usage guidelines, aimed at reducing the learning curve, improving usability, and better integrating the technique into team workflows as reported in Corrêa, 2025.
- A characterization of architectural uncertainties managed by these teams, describing their sources, quality attributes, and evolution, as captured through Arch-Stage as reported in Corrêa, 2025.

- Reflections on the implications of these findings for supporting distributed architectural work in agile and continuous development settings, connecting the ArchHypo experience to broader concerns about tacit knowledge and tool support in software architecture as reported in Souza *et al.*, 2019; Betts *et al.*, 2024.

1.7 Research Structure

The remainder of this work is organized as follows.

Chapter 2 presents the background and related work. It introduces key concepts in software architecture and architectural uncertainty, summarizes the ArchHypo technique and previous empirical studies about it, describes the initial design of the Arch-Stage tool and the undergraduate thesis that developed it, and revisits the main findings from systematic studies on deriving architecture from requirements and on tool support in this area by Souza *et al.*, 2019.

Chapter 4 details the research design and methods adopted in this dissertation. It describes the industrial context in which the study was conducted, the participating teams and their product, the data collection instruments and procedures, and the analysis methods used to answer the research questions.

The chapter reporting empirical results presents how teams used Arch-Stage and ArchHypo in practice, the architectural hypotheses and uncertainties they managed, and the perceived benefits and challenges from the perspective of different roles.

The chapter discussing refinements details the changes made to Arch-Stage and its usage guidelines based on empirical findings. It explains the rationale for the changes, illustrates how they address specific issues observed in the field, and reflects on remaining limitations and opportunities for future improvement.

Finally, the conclusion chapter summarizes the main contributions, discusses implications for practitioners and researchers, and outlines directions for future work on tool support for architectural uncertainty management and distributed architecture decision-making.

Chapter 2

Background

This chapter establishes the conceptual foundations for the empirical investigation carried out in this thesis. It begins with a concise overview of software architecture and architectural uncertainty in the context of scaled products, then introduces hypotheses engineering and the ArchHypo technique in full technical detail, describes the Arch-Stage tool, and concludes with a summary of findings from Souza et al.'s mapping study on the gap between architectural derivation methods and practical tool support.

2.1 Software Architecture and Architectural Uncertainty

Software architecture refers to the high-level structure of a software system, encompassing its components, their relationships, and the design principles that guide its evolution Bass *et al.*, 2021. In organizations that maintain scaled products—systems developed and operated by multiple teams—architectural decisions shape not only technical quality attributes such as performance, reliability, and maintainability, but also the ability of teams to work independently and deliver changes at speed Skelton and Pais, 2019.

Richards and Ford define software architecture as the structure of a system combined with the characteristics it must support, the principles and decisions guiding its design, and the design decisions themselves Richards and Ford, 2020. This definition emphasizes that architecture is a living body of reasoning, not a frozen blueprint. Perry and Wolf's earlier tripartite model Perry and Wolf, 1992 decomposed architecture into elements (the processing, data, and connecting constituents), form (the weighted relationships among those elements), and rationale (the set of constraints that justify the structure). Together, these accounts foreground the role of *uncertainty*: the rationale behind any architectural choice is valid only relative to the information available at the time of the decision, and that information is always incomplete.

Architectural uncertainty arises whenever teams must make structural decisions under incomplete information. This is a normal condition in continuous-delivery environments where business requirements, user behavior, and technology options are only partially

known. When uncertainty is left implicit, teams either defer decisions indefinitely (accumulating technical debt) or make premature commitments that constrain future options. Both outcomes slow delivery and degrade system quality Foote and Yoder, 1999.

The concept of "fast flow," popularized by the Team Topologies framework, emphasizes that organizations should design both their teams and their architecture to enable a continuous, rapid stream of software changes. Achieving fast flow presupposes that teams can make local architectural decisions without waiting for a central architect. This in turn demands that architectural uncertainties be made explicit and that lightweight mechanisms exist for teams to assess, prioritize, and revisit them as evidence accumulates.

2.2 Hypotheses Engineering and ArchHypo

Hypotheses engineering is an approach to managing technical uncertainty by formulating architectural decisions as testable hypotheses rather than settled facts. Instead of asserting that a particular technology or pattern *will* satisfy a quality requirement, teams state that it *might* do so and then design experiments to gather evidence.

The ArchHypo technique, proposed by Silva et al., operationalizes this idea for software architecture E. Silva, J. Melegati, Silveira, *et al.*, 2025. Grounded in Design Science Research (DSR) and evolved from a body of patterns for agile architecture first disseminated at conferences and through corporate training E. Silva, J. Melegati, Wang, *et al.*, 2024, ArchHypo provides a structured vocabulary and a set of practices that fit within iterative development workflows.

2.2.1 Core Concepts

An **architectural hypothesis** is a falsifiable statement that captures an uncertainty relevant to the design of a software architecture. It is important to distinguish the concept from adjacent notions:

- **Hypothesis vs. risk.** A risk involves a known event with estimable probability and impact. An architectural hypothesis, by contrast, stems from a *lack of information* sufficient to make a decision. The distinguishing feature is the epistemic gap, not the probability of an outcome E. Silva, J. Melegati, Silveira, *et al.*, 2025.
- **Hypothesis vs. requirement.** A requirement specifies what the system must do or how well it must do it. A hypothesis may be triggered by a requirement (e.g., a quality-attribute goal that cannot yet be confirmed as achievable), but the hypothesis captures the uncertainty about the path to satisfying that requirement, not the requirement itself.
- **Hypothesis vs. architectural decision.** An architectural decision resolves a design choice. A hypothesis precedes and informs decisions: it identifies what must be learned or validated before a decision can responsibly be made E. Silva, J. Melegati, Silveira, *et al.*, 2025.

Hypotheses arise from two principal sources: (a) *requirements*—when quality attribute targets are poorly specified or the team lacks information about whether a given approach

can satisfy them; and (b) *solutions*—when the consequences of an existing or candidate architectural solution are unknown.

2.2.2 Assessment

Each hypothesis is evaluated along two independent dimensions:

1. **Uncertainty level.** How far the team is from being able to refute or accept the hypothesis. Crucially, uncertainty level is *not* equivalent to probability. A hypothesis that is almost certainly true *and* a hypothesis that is almost certainly false both represent *low* uncertainty, because in either case the team already has enough information to act. High uncertainty means the team genuinely does not know which way the evidence will fall. Both dimensions can be assessed using a 3-point or 5-point qualitative scale; assessments are made independently and relatively, not on absolute probability E. Silva, J. Melegati, Silveira, *et al.*, 2025.

2. **Impact level.** The potential consequences—in terms of rework, schedule, cost, or quality degradation—if the hypothesis is invalidated. A high uncertainty combined with a high impact signals that a hypothesis demands urgent attention.

2.2.3 Technical Plans

For hypotheses with significant uncertainty or impact, teams formulate a **technical plan**: a set of concrete actions whose goal is to (a) accept or refute the hypothesis, (b) reduce its uncertainty level, (c) reduce its potential impact, or (d) define criteria under which the decision can responsibly be deferred E. Silva, J. Melegati, Silveira, *et al.*, 2025. ArchHypo identifies the following plan strategies:

- **Architectural Spike.** A time-boxed, isolated experiment to explore a technology or design option without coupling the result to the production codebase. The spike produces information rather than deliverable functionality.

- **Tracer Bullet.** A thin, end-to-end implementation within the actual system, exercising all architectural layers to validate an integration or performance characteristic under realistic conditions.

- **Software Analytics.** Using monitoring, profiling, or usage data already collected from the running system to gather evidence about a hypothesis without additional development effort.

- **Modularity / Flexibility.** Restructuring the affected subsystem to reduce coupling or increase replaceability, thereby lowering the *impact* of the hypothesis rather than resolving its uncertainty. This strategy is useful when full resolution is premature but protection against a bad outcome is warranted.

- **Guideline Implementation.** Tailoring the development process—for example, adopting a coding standard, a review protocol, or a testing policy—so that the team systematically generates evidence about the hypothesis as a by-product of normal work. This was the most frequently used strategy in the TSE evaluation E. Silva, J. Melegati, Silveira, *et al.*, 2025.

- **Architectural Trigger.** A condition-based deferral in which the team agrees to act on the hypothesis only when a specific observable event occurs (e.g., when the number of concurrent users exceeds a threshold, or when a performance regression appears in monitoring). Until the trigger fires, the hypothesis is deliberately deferred.
- **Agile Landing Zones.** Adopting architectural patterns that explicitly preserve flexibility, creating "landing zones" into which future decisions can be inserted without major disruption, regardless of which direction the evidence eventually points.

2.2.4 Lifecycle

Hypotheses progress through a defined lifecycle: they begin in an *open* state and transition to either *validated* or *invalidated* as evidence from technical plans accumulates. The responsible-moment principle—act when you have enough information, neither too early nor too late—guides when a hypothesis should be resolved rather than monitored further.

2.2.5 Empirical Evaluations

IEEE Software evaluation (Catch Solve startup). The first evaluation of ArchHypo in a real operating environment was reported in an IEEE Software article by Silva et al. E. Silva, J. Melegati, Wang, *et al.*, 2024. The setting was Catch Solve, a startup offering web testing and monitoring services. Four hypotheses were formulated and tracked over approximately ten months, covering availability, reusability of test templates, usability, and scalability of virtual machines. The study showed that naming and tracking uncertainties explicitly helped the technical lead decide when to invest in architectural improvements versus when to wait, and validated the overall structure of the ArchHypo approach.

TSE evaluation (PropSEP at Jetsoft). The larger and more controlled evaluation, published in the IEEE Transactions on Software Engineering E. Silva, J. Melegati, Silveira, *et al.*, 2025, examined the application of ArchHypo within Jetsoft, a Brazilian software development company, on the PropSEP project—the budget proposal system of the Planning and Management Secretariat of the State of São Paulo. PropSEP is mission-critical: the approved budget it processes amounts to approximately 317 billion BRL. The evaluation ran across six sprints of seven business days each (May–August 2022) and involved eight participants (P1–P8), all holding computer science degrees with certifications in Oracle, WebSphere, or GeneXus. Ten hypotheses were identified, spanning six quality attributes (performance, security, reliability, flexibility, usability, and team productivity), and seven distinct types of architectural practice were employed in the resulting technical plans.

The study produced nine key findings:

1. Some hypotheses carried uncertainty that was monitored throughout the entire project duration without reaching full resolution, demonstrating that the technique accommodates long-horizon uncertainties.
2. ArchHypo enabled the team to avoid upfront architectural design, distributing structural decisions across iterations as evidence became available.

3. Participants reported improvements to the development process in terms of safety, transparency, and predictability.
4. Hypotheses spanning different quality attributes were handled using qualitatively different technical strategies, illustrating the creativity the technique affords.
5. Guideline Implementation (process tailoring) emerged as the most frequently employed strategy across all technical plans.
6. The team collectively recognized that managing hypotheses improved the quality of their architectural decision-making.
7. Post-release evidence pointed to a positive impact on product quality: no issues related to the managed hypotheses were detected after deployment.
8. **Changes in the team's way of working constituted a significant adoption obstacle.** Mixing architectural and hypothesis-related tasks with functional development created estimation difficulties, and participants described the additional work as not feeling directly connected to product delivery.
9. **All eight participants unanimously considered the technique hard to learn.** Specific difficulties included: mapping architectural concerns to the hypothesis format, formulating appropriate technical plan actions, articulating the assumptions behind a hypothesis, and understanding hypotheses as the primary driver of architectural action. No participant felt prepared to apply ArchHypo without continued expert support after the initial workshop.

Findings 8 and 9 directly motivate the tooling contribution of this thesis: a tool that embeds ArchHypo scaffolding in the daily engineering workflow, reducing the cognitive overhead of manual application.

2.3 Arch-Stage: The Initial Tool

Arch-Stage (originally named HypoStage) is an open-source Backstage plugin developed as an undergraduate capstone project by Pedro Henrique Mariano Corrêa, supervised by the author of this thesis and co-supervised by Prof. Paulo Meirelles Corrêa, 2025. Its purpose is to operationalize ArchHypo within an Internal Developer Portal (IDP), embedding hypothesis management into the engineering workflow.

The initial version of Arch-Stage, corresponding to the code up to commit `ceee509` in the `hypo-stage` repository,[1] provides the following capabilities:

- **Hypothesis management**: creating, editing, listing, and deleting architectural hypotheses, each linked to a Backstage catalog entity (e.g., a service, component, or system). Integration with the Backstage catalog means that hypotheses are automatically associated with the software entities they affect, without requiring separate registration.

[1] https://github.com/ArchHypo/hypo-stage

- **Uncertainty and impact assessment**: rating each hypothesis on 5-point Likert scales for both uncertainty level and impact level, with free-text rationale fields. The use of a 5-point scale rather than a binary or 3-point scale provides finer-grained prioritization while remaining tractable for teams without formal risk management backgrounds.
- **Technical planning**: attaching one or more technical plan items to a hypothesis, specifying action type (spike, tracer bullet, trigger, etc.), description, target date, expected outcome, and documentation links.
- **Visualization**: a chart plotting hypotheses on an uncertainty-vs-impact plane to support prioritization, making the relative urgency of hypotheses visible at a glance.
- **Status lifecycle**: tracking each hypothesis through the defined lifecycle states (open → validated or invalidated), preserving the history of status transitions to support retrospective analysis.

By integrating with Backstage, Arch-Stage leverages the portal's service catalog, so that hypotheses are automatically associated with the software entities they affect. This design reduces context-switching: developers manage architectural hypotheses in the same environment where they consult documentation, inspect service health, and access CI/CD pipelines.

Known limitations. The initial tool design assumed familiarity with the ArchHypo vocabulary, which poses a barrier for teams new to hypothesis engineering. The case study in the capstone thesis reconstructed the Catch Solve startup scenario digitally but did not involve real-time use by a team. Furthermore, the tool lacked features for collaborative assessment (e.g., multiple raters per hypothesis) and for tracking the evolution of assessments over time. These gaps motivate the empirical evaluation and refinement effort in this thesis.

2.4 Deriving Architecture from Requirements: The Tooling Gap

The systematic mapping study by Souza, Moreira, and Goulão Souza *et al.*, 2019 examined the existing body of work on methods for deriving architectural models from requirements specifications. Starting from 2,575 candidate papers, the authors applied a PICOC (Population, Intervention, Comparison, Outcome, Context) search structure and a quality assessment checklist—composed of general criteria (weighted at 25%) and method-specific criteria (weighted at 75%)—to select 39 high-quality primary studies for detailed analysis. Their analysis employed the NIMSAD framework to characterize the structure of each reviewed method.

Among their key findings directly relevant to this thesis:

1. **Dependence on tacit knowledge.** Every reviewed method relied, to some degree, on the architect's experience and intuition. No method provided a fully automated or guided derivation path accessible to less-experienced practitioners.

2. **Limited support for less-experienced practitioners.** Methods assumed a skilled architect capable of navigating trade-offs among quality attributes. Teams without senior architectural expertise had little actionable guidance.
3. **Weak or absent tool support.** Although some tools were identified—most notably QuaDAI and Monarch—they addressed only fragments of the derivation process and offered only partial coverage of the architectural reasoning cycle. The mapping found no comprehensive tool integrating hypothesis formulation, assessment, and tracking within a team's daily workflow.
4. **Insufficient evaluation.** Many proposed methods lacked rigorous empirical validation, relying on toy examples or author-run case studies rather than independent evaluations with real teams in production environments.

Souza et al. also noted that 76.9% of the 39 primary studies addressed functional requirements, with a significant proportion also treating non-functional requirements; yet the methods for handling quality attributes—precisely the domain where architectural uncertainty is most consequential—were among the least well supported by tools or empirical evidence.

These findings reinforce the motivation for this thesis on two fronts. First, they confirm the existence of a "tooling void" in which architectural management remains disconnected from the daily engineering workflow. Second, they underscore the need for empirical evaluations with real teams—precisely the kind of evaluation this thesis aims to provide for Arch-Stage and ArchHypo.

Chapter 3

Related Work

This chapter provides a contextual synthesis of literature relevant to the empirical evaluation of Arch-Stage. Rather than presenting a standalone systematic review, it draws on the author's earlier qualification work—which included a tertiary systematic mapping study on software architecture for startups and a multivocal literature review on centralization and decentralization of architectural decision-making—and repositions those findings as narrative context for the present thesis.

3.1 Centralization and Decentralization of Architectural Decisions

A recurring theme in both academic and practitioner literature is the tension between centralizing architectural authority in a few experienced individuals and distributing it across development teams. Organizations pursuing fast flow and team autonomy increasingly favor decentralized models, yet several studies report that without explicit governance mechanisms, decentralization leads to inconsistent decisions and architectural drift.

The author's qualification examination explored this tension through two complementary lenses. The first chapter of the qualification presented a tertiary systematic mapping study surveying the existing landscape of secondary studies on software architecture for startup and scale-up organizations, identifying recurring themes in how early-stage teams approach architecture governance under resource and time constraints. The second chapter conducted a multivocal literature review that synthesized both academic publications and grey literature (practitioner blogs, conference talks, and industry reports), covering three interrelated perspectives: deriving software architecture from requirements, the emerging socio-technical architecture trend (which situates architectural decisions within organizational and team structures), and the tension between centralized and decentralized decision-making.

The multivocal review surfaced a framing that the qualification termed the "paradox of software architecture decision-making": industry advocates strongly for decentralized decisions to enable team autonomy and fast flow, yet the methods and tools available

for architectural decision-making depend heavily on the tacit knowledge of experienced architects. Decentralization thus risks transferring decision responsibility to practitioners who lack the conceptual vocabulary or methodological support to exercise it effectively. The qualification committee, however, found this "paradox" framing ultimately unproductive as a primary research framing: the tension it described reflects a recognized design trade-off rather than a genuine logical contradiction, and building a thesis around resolving a paradox risked over-extending the scope of the contribution. This thesis consequently does not employ that framing; instead, it focuses on the concrete, tractable problem of providing tool support for one well-defined uncertainty management technique.

The third chapter of the qualification proposed a questionnaire-based research design to investigate how software engineering teams actually navigate architectural decisions in distributed settings. The proposed questionnaire targeted four dimensions: demographic and organizational context, knowledge of modern software architecture practices, collaboration and decision-making patterns within and across teams, and specific challenges experienced in decentralized architectural decision-making. While this research design would have yielded broad survey data, the qualification committee assessed the scope as too wide and the proposed SMS and multivocal review as insufficient as primary thesis contributions without a concrete artifact or evaluation at the center. The present thesis responds to that feedback by reorienting around the evaluation and refinement of Arch-Stage, a concrete tool artifact, rather than a survey of industry practice.

3.2 Tool Support for Architectural Decision-Making

Several tools and platforms have been proposed to support architectural decision-making:

- **ADR tools** (e.g., adr-tools, Log4brains) focus on recording and versioning Architecture Decision Records (ADRs). They capture the *what* and *why* of decisions but do not model uncertainty, impact, or technical plans for hypotheses that are not yet decisions. ADRs document settled choices; they are not designed to manage the pre-decision state of uncertainty that ArchHypo addresses.
- **Architecture analysis tools** (e.g., ATAM-based workshops, trade-off analysis methods) provide structured evaluation of architectural alternatives but are typically heavyweight, workshop-based activities rather than continuous, team-embedded processes. Their value lies in one-shot or periodic analysis sessions, not in the ongoing, iterative management of uncertainty throughout a product's lifecycle.
- **Internal Developer Portals** such as Backstage, Port, and Cortex offer a catalog-centric view of services and infrastructure. While they improve discoverability and documentation, none of these platforms natively supports hypothesis engineering or architectural uncertainty management. They provide the integration surface that Arch-Stage exploits, but leave the hypothesis engineering layer entirely absent.

Arch-Stage differentiates itself by being explicitly designed around the ArchHypo lifecycle: from hypothesis elicitation through assessment, technical planning, and resolution. Its integration with Backstage places this lifecycle within the developer's daily

tooling environment. No existing tool, to the best of the author's knowledge, provides this combination.

3.3 Empirical Studies of Architectural Practices in Industry

Industry-focused empirical studies on architectural practices remain relatively scarce compared with tool proposals and method descriptions. Among the most relevant:

- The ArchHypo evaluations by Silva et al. E. Silva, J. Melegati, Silveira, *et al.*, 2025; E. Silva, J. Melegati, Wang, *et al.*, 2024 constitute the closest antecedents to this work. They demonstrated the value of hypothesis engineering in two distinct settings—a web-testing startup and a mission-critical government budget system—but relied entirely on manual application with no tool support. This thesis introduces the first evaluation of ArchHypo in a tool-mediated setting.
- Studies on architectural governance in agile organizations (e.g., the work on SAFe and Spotify-model variants) document how companies structure roles and ceremonies for architecture, but rarely examine explicit uncertainty-management techniques.
- The multivocal review conducted during the author's qualification surfaced practitioner reports (blog posts, conference talks, white papers) describing ad-hoc approaches to architectural uncertainty. These reports corroborate the need for structured methods but, being grey literature, carry weaker evidential value.

This thesis contributes to the body of empirical evidence by evaluating a tool-supported application of ArchHypo with multiple teams, addressing a gap in both the ArchHypo literature (which has not yet studied tool-mediated use) and the broader architectural-practices literature (which lacks studies of hypothesis-engineering tools in scaled settings).

Chapter 4

Research Design

This chapter describes the empirical approach adopted to evaluate and refine the Arch-Stage tool. The design follows a mixed-methods strategy aligned with the thesis objective: understanding how software engineering teams use a tool for managing architectural hypotheses in a real-world setting, and identifying refinements needed to improve its practical utility.

4.1 Empirical Context

The study takes place within **CompanyX**, an organization that develops and operates **ProductY**, a scaled software product serving a significant user base. Multiple software engineering teams collaborate on different subsystems of ProductY, each responsible for distinct functional areas but sharing cross-cutting architectural concerns such as scalability and reliability.

Teams and participants. At least two teams are involved in the study, representing varying levels of architectural experience—from teams with a senior architect to teams where architectural responsibility is distributed informally among developers.

Tool deployment. Arch-Stage was deployed within CompanyX's Backstage instance, making it accessible alongside existing developer portal features. Teams were introduced to the ArchHypo concepts and the Arch-Stage interface through a short onboarding session before beginning their use of the tool.

4.2 Research Approach

The research adopts a **mixed-methods** approach combining qualitative and quantitative data sources. This choice is motivated by the exploratory nature of the study—there are no prior studies of tool-supported ArchHypo use—and by the need to capture both behavioral patterns (how teams use the tool) and perceptual data (how teams experience the tool).

The study can be characterized as an *embedded case study* with multiple units of analysis (the participating teams), following guidelines from Yin and Runeson et al. for empirical software engineering research.

The design is informed by the approach taken in the TSE evaluation of ArchHypo E. Silva, J. Melegati, Silveira, *et al.*, 2025, in which a participant-observer role was used to collect multi-source evidence: Kanban board history, user stories, incident reports, the ArchHypo tracking spreadsheet, relevant e-mails, and a questionnaire administered to all team members. That study demonstrated the value of combining artifact-based and perceptual evidence for understanding both how a technique is applied and how participants experience it. The present thesis extends this methodological approach to the tool-mediated setting, where Arch-Stage itself becomes both a data source (its stored artifacts) and an object of evaluation (its usability and fit).

Critically, the TSE evaluation used manual spreadsheets as its primary tracking medium. This thesis evaluates a qualitatively different mode of application: one in which the tool scaffolds hypothesis management, potentially reducing the cognitive overhead and process-change friction identified in Findings 8 and 9 of that study.

4.3 Data Sources

Three primary data sources are planned:

1. **Arch-Stage artifacts.** The hypotheses, assessments, and technical plans recorded by teams in Arch-Stage constitute the principal data set. For each hypothesis, the following attributes are available: statement, quality attribute, source type, uncertainty rating, impact rating, technical plan items (type, description, target date, status), and lifecycle status changes over time. Unlike the spreadsheet-based data of the TSE study, these artifacts are captured in a structured database, enabling systematic extraction and longitudinal analysis.

2. **Questionnaires.** Short questionnaires administered to team members after a defined usage period (e.g., one sprint cycle or one month) to capture perceptions of usability, perceived value, and difficulties encountered.

3. **Semi-structured interviews.** Interviews with a subset of participants (e.g., team leads, developers who created hypotheses, developers who did not) to explore themes emerging from the artifact and questionnaire data in greater depth.

Optionally, the study may include:

- **Observation of meetings.** Attending architecture discussions or sprint-planning sessions where Arch-Stage is referenced, to observe how the tool mediates conversation.

- **Usage logs.** If available, server-side logs of Arch-Stage interactions (page views, edits, time spent) could provide objective usage metrics.

4.4 Analysis Plan

Artifact analysis. Hypotheses and technical plans will be coded thematically to identify recurring types of architectural uncertainty (e.g., technology-fit uncertainties, integration uncertainties, performance uncertainties) and patterns in how teams formulate and resolve them. Uncertainty and impact ratings will be analyzed descriptively (distributions, medians) and, if sample size permits, compared across teams.

Questionnaire analysis. Likert-scale responses will be summarized with descriptive statistics. Open-ended responses will be coded using the same thematic framework applied to the artifact data.

Interview analysis. Interview transcripts will be analyzed using thematic coding, following the six-phase approach of Braun and Clarke. Codes will be organized into themes addressing each research question (tool usage patterns, needed refinements, types of uncertainty).

Triangulation. Findings from the three data sources will be triangulated to strengthen the validity of conclusions. Where artifact data and interview data converge, confidence in the finding increases; where they diverge, the divergence itself becomes a finding requiring interpretation.

4.5 Ethical Considerations

All participants will be asked for informed consent before data collection begins. Company and product names will be anonymized in all publications. Individual participant data (questionnaire responses, interview transcripts) will be stored securely and accessible only to the research team.

4.6 Alignment with Qualification Feedback

This research design responds to the main feedback received during the author's qualification examination:

- **Narrowed scope**: the study focuses on specific teams and a concrete tool rather than broad surveys of software architecture trends. The qualification committee found the earlier scope—encompassing both a tertiary SMS and a multivocal review as principal contributions—too wide for a master's thesis.
- **Rejection of the "paradox" framing**: the "paradox of SA decision-making" terminology used in the qualification was judged unproductive by the committee, as it overstated the nature of what is in practice a design trade-off. This thesis does not employ that framing.
- **Limited research questions**: three focused questions replace the earlier, broader set.

- **Concrete artifact**: Arch-Stage serves as the tangible object of evaluation, grounding the research in observable behavior and artifacts. The qualification's proposed questionnaire-based survey was repositioned as contextual motivation rather than a primary contribution.
- **Empirical rigor**: the mixed-methods design, triangulation plan, and explicit ethical considerations aim for a level of rigor appropriate for a master's thesis in empirical software engineering. By modeling the data collection strategy on the TSE study's multi-source approach while extending it to the tool-mediated context, the design offers a coherent methodological lineage.

Appendix A

Research Instruments

A.1 Questionnaire

A.2 Interview Guide

A.3 Informed Consent Form

References

[Bass *et al.* 2021] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. 4th ed. Addison-Wesley, 2021 (cit. on p. 7).

[Betts *et al.* 2024] Thomas Betts, Blanca García Gil, Eran Stiller, Daniel Bryant, and Rafal Gancarz. *InfoQ Software Architecture and Design Trends Report — 2024*. https://www.infoq.com/articles/architecture-trends-2024/. Accessed: 23 March 2026. 2024 (cit. on pp. 1, 2, 4, 6).

[Corrêa 2025] Pedro Henrique Mariano Corrêa. *ArchHypo Tooling: Enabling Practical Hypothesis Engineering for Software Architecture*. Capstone Project Report (Bachelor), Institute of Mathematics and Statistics, University of São Paulo. Supervised by José G. Lima Neto; co-supervised by Paulo Meirelles. 2025 (cit. on pp. 3–5, 11).

[Foote and Yoder 1999] Brian Foote and Joseph Yoder. "Big ball of mud". *Pattern Languages of Program Design* 4 (1999), pp. 654–692 (cit. on p. 8).

[Perry and Wolf 1992] Dewayne E. Perry and Alexander L. Wolf. "Foundations for the study of software architecture". *ACM SIGSOFT Software Engineering Notes* 17.4 (1992), pp. 40–52. doi: 10.1145/141874.141884 (cit. on pp. 1, 7).

[Richards and Ford 2020] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. isbn: 9781492043454 (cit. on p. 7).

[E. Silva, J. Melegati, Silveira, *et al.* 2025] Eduardo Silva, Jorge Melegati, Moises Silveira, *et al.* "ArchHypo: managing software architecture uncertainty using hypotheses engineering". *IEEE Transactions on Software Engineering* 51.2 (2025), pp. 481–499. doi: 10.1109/TSE.2024.3520477 (cit. on pp. 8–10, 17, 20).

[E. Silva, J. Melegati, Wang, *et al.* 2024] Eduardo Silva, Jorge Melegati, Xiaofeng Wang, and Paulo Meirelles. "Using hypotheses to manage technical uncertainty and architecture evolution in a software start-up". *IEEE Software* 41.4 (2024), pp. 78–87. doi: 10.1109/MS.2024.3383628 (cit. on pp. 8, 10, 17).

[K. Silva, J. Melegati, *et al.* 2025] Kelson Silva, Jorge Melegati, *et al.* "Archhypo: managing software architecture uncertainty using hypotheses engineering". *IEEE Transactions on Software Engineering* 51.2 (2025), pp. 430–446. doi: 10.1109/TSE.2024.3520477 (cit. on pp. 1–4).

[K. Silva, J. A. Melegati, *et al.* 2024] Kelson Silva, Jorge A. Melegati, Xiaofeng Wang, Mauricio Ferreira, and Eduardo Guerra. "Using hypotheses to manage technical uncertainty and architecture evolution in a software start-up". *IEEE Software* 41.4 (2024), pp. 7–13. doi: 10.1109/MS.2024.3383628 (cit. on pp. 1–4).

[Skelton and Pais 2019] Matthew Skelton and Manuel Pais. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press, 2019 (cit. on p. 7).

[Souza *et al.* 2019] Eric Souza, Ana Moreira, and Miguel Goulão. "Deriving architectural models from requirements specifications: a systematic mapping study". *Information and Software Technology* 109 (2019), pp. 26–39. doi: 10.1016/j.infsof.2019.01.004 (cit. on pp. 1–4, 6, 12).